
Table of Contents

=====	1
--- Configuration -----	2
--- NOTE: Simplified CEC'2005 functions -----	2
--- CEC'2005 Function Definitions -----	2
--- Algorithm hyper-parameters -----	2
--- Main Experiment Loop -----	3
GA — 15 runs, record final value AND convergence curve	3
PSO — 15 runs	4
Store	4
--- Results Table -----	4
--- Figure 1: Box Plots -----	5
--- Figure 2: Convergence Curves (mean $\pm$ std) -----	6
--- Figure 3: Per-Run Value Scatter -----	8
=====	9
=====	9
=====	12
=====	12
=====	14
=====	14

STW7085CEM - Advanced Machine Learning
Task 2, Part 3 - CEC'2005 Benchmark Functions
GA vs PSO on F2 (Schwefel 1.2) and F8 (Ackley)
D = 2 and D = 10 | 15 independent runs each
No Toolboxes required – pure base-MATLAB

CHANGES vs previous version:

1. run_custom_ga / run_custom_pso now RETURN convergence history (best value per generation) – required for "convergence graphs" in the assignment brief.
2. Convergence plot added: mean $\pm$ std band over 15 runs.
3. PSO now has velocity clamping ($V_{max} = 0.2 \cdot \text{range}$).
4. Crossover uses uniform crossover (not single-point) so D=2 crossover is non-trivial.
5. Note about simplified functions (no CEC data files).

```
clear; clc; close all;
```

---- Configuration -----

```
N_RUNS    = 15;
D_VALUES  = [2, 10];
```

---- NOTE: Simplified CEC'2005 functions

The real CEC'2005 F2 and F8 require loading shift vectors and rotation matrices from the official data files (available from the CEC'2005 technical report). This script uses the base (unshifted, unrotated) versions, which differ only in the constant offset (global optimum location is at origin instead of the shifted point). The relative performance of GA vs PSO is unaffected. Report this as a simplification in your written submission.

---- CEC'2005 Function Definitions

F2: Schwefel's Problem 1.2 (unimodal, non-separable) $f(x) = \sum_{i=1}^D (\sum_{j=1}^i x_j)^2$ Shifted global optimum = -450

```
F2_bias = -450;
f2 = @(x, D) sum( arrayfun(@(i) sum(x(1:i)).^2, 1:D) ) + F2_bias;
```

```
% F8: Ackley's Function (multimodal, many local minima)
%      Shifted global optimum = -140
```

```
F8_bias = -140;
f8 = @(x, D) -20*exp(-0.2*sqrt((1/D)*sum(x.^2))) ...
    - exp((1/D)*sum(cos(2*pi*x))) ...
    + 20 + exp(1) + F8_bias;
```

```
functions    = {f2, f8};
func_names   = {'F2 Schwefel 1.2', 'F8 Ackley'};
func_ids     = [2, 8];
func_optima  = [-450, -140];
func_ranges  = {[ -100 100], [ -32 32]};
```

---- Algorithm hyper-parameters

GA

```
ga_popsiz = 100;
ga_maxgen = 500;
ga_pcross = 0.8;
ga_pmut   = 0.02;
ga_elite  = 2;
```

```
% PSO
pso_swarm = 100;
```

```

=====
=====
=====
pso_maxiter = 500;
pso_w       = 0.729;    % inertia weight
pso_c1      = 1.494;    % cognitive coefficient
pso_c2      = 1.494;    % social coefficient
pso_vmax_f  = 0.2;      % Vmax as fraction of search range

```

---- Main Experiment Loop

```

=====

results = struct();

for fi = 1:length(functions)
    func      = functions{fi};
    fname     = func_names{fi};
    fid       = func_ids(fi);
    frange    = func_ranges{fi};
    f_opt     = func_optima(fi);

    for di = 1:length(D_VALUES)

        D      = D_VALUES(di);
        lb     = repmat(frange(1), 1, D);
        ub     = repmat(frange(2), 1, D);
        MaxFE  = 10000 * D;

        actual_ga_gen  = min(ga_maxgen, floor(MaxFE / ga_popsize));
        actual_pso_iter = min(pso_maxiter, floor(MaxFE / pso_swarm));

        fprintf('\n=== %s | D=%d | MaxFE=%d | GA gens=%d | PSO iters=%d
===\n', ...
            fname, D, MaxFE, actual_ga_gen, actual_pso_iter);

        === F2 Schwefel 1.2 | D=2 | MaxFE=20000 | GA gens=200 | PSO iters=200 ===

        === F2 Schwefel 1.2 | D=10 | MaxFE=100000 | GA gens=500 | PSO iters=500 ===

        === F8 Ackley | D=2 | MaxFE=20000 | GA gens=200 | PSO iters=200 ===

        === F8 Ackley | D=10 | MaxFE=100000 | GA gens=500 | PSO iters=500 ===

```

GA — 15 runs, record final value AND convergence curve

```

ga_vals = zeros(N_RUNS, 1);
ga_conv = zeros(N_RUNS, actual_ga_gen); % <-- convergence history

for r = 1:N_RUNS

```

```

=====
=====
=====
rng(r);
fobj = @(x) func(x, D);
[ga_vals(r), ga_conv(r,:)] = run_custom_ga( ...
    fobj, D, lb, ub, ga_popsizes, actual_ga_gen, ...
    ga_pcross, ga_pmut, ga_elite);
fprintf(' GA run %2d: %.4f\n', r, ga_vals(r));
end

```

PSO — 15 runs

```

pso_vals = zeros(N_RUNS, 1);
pso_conv = zeros(N_RUNS, actual_pso_iter); % <-- convergence history
Vmax      = pso_vmax_f * (frange(2)-frange(1));

for r = 1:N_RUNS
    rng(r + 100);
    fobj = @(x) func(x, D);
    [pso_vals(r), pso_conv(r,:)] = run_custom_pso( ...
        fobj, D, lb, ub, pso_swarm, actual_pso_iter, ...
        pso_w, pso_c1, pso_c2, Vmax);
    fprintf(' PSO run %2d: %.4f\n', r, pso_vals(r));
end

```

Store

```

key = sprintf('F%d_D%d', fid, D);
results.(key).ga_vals = ga_vals;
results.(key).pso_vals = pso_vals;
results.(key).ga_conv = ga_conv;
results.(key).pso_conv = pso_conv;
results.(key).func_name = fname;
results.(key).fid = fid;
results.(key).D = D;
results.(key).optimum = f_opt;
results.(key).n_ga_gen = actual_ga_gen;
results.(key).n_pso_it = actual_pso_iter;

end
end

```

---- Results Table -----

```

fprintf('\n\n%-22s %-4s %-6s %12s %12s %12s %12s\n', ...
    'Function', 'D', 'Algo', 'Mean', 'Std', 'Best', 'Worst');
fprintf('%s\n', repmat('-', 1, 82));

keys = fieldnames(results);
for k = 1:length(keys)
    r = results.(keys{k});
    fprintf('%-22s %-4d %-6s %12.4f %12.4f %12.4f %12.4f\n', ...
        r.func_name, r.D, 'GA', ...
        mean(r.ga_vals), std(r.ga_vals), min(r.ga_vals), max(r.ga_vals));
end

```

```

=====
=====
===
fprintf('%-22s %-4d %-6s %12.4f %12.4f %12.4f %12.4f\n', ...
    '', r.D, 'PSO', ...
    mean(r.pso_vals), std(r.pso_vals), min(r.pso_vals), max(r.pso_vals));
fprintf('%-22s %-4d %-6s %12.4f (global optimum)\n', ...
    '', r.D, 'OPT', r.optimum);
fprintf('%s\n', repmat('-',1,82));
end

```

Function		D	Algo	Mean	Std	
Best	Worst					

F2 Schwefel 1.2		2	GA	-449.1685	1.1379	-449.9693
-445.7167		2	PSO	-450.0000	0.0000	-450.0000
-450.0000		2	OPT	-450.0000	(global optimum)	

F2 Schwefel 1.2		10	GA	-211.2208	107.7509	
-384.5450	-24.6896	10	PSO	-450.0000	0.0000	-450.0000
-450.0000		10	OPT	-450.0000	(global optimum)	

F8 Ackley		2	GA	-139.1262	0.6818	-139.9389
-137.7995		2	PSO	-140.0000	0.0000	-140.0000
-140.0000		2	OPT	-140.0000	(global optimum)	

F8 Ackley		10	GA	-139.6832	0.1107	-139.8407
-139.4189		10	PSO	-140.0000	0.0000	-140.0000
-140.0000		10	OPT	-140.0000	(global optimum)	

---- Figure 1: Box Plots -----

```

figure('Name','Box Plots','NumberTitle','off','Position',[50 50 1200 480]);

label_list = {};
data_matrix = [];
for fi = 1:2
    fid = func_ids(fi);
    for di = 1:length(D_VALUES)
        D = D_VALUES(di);

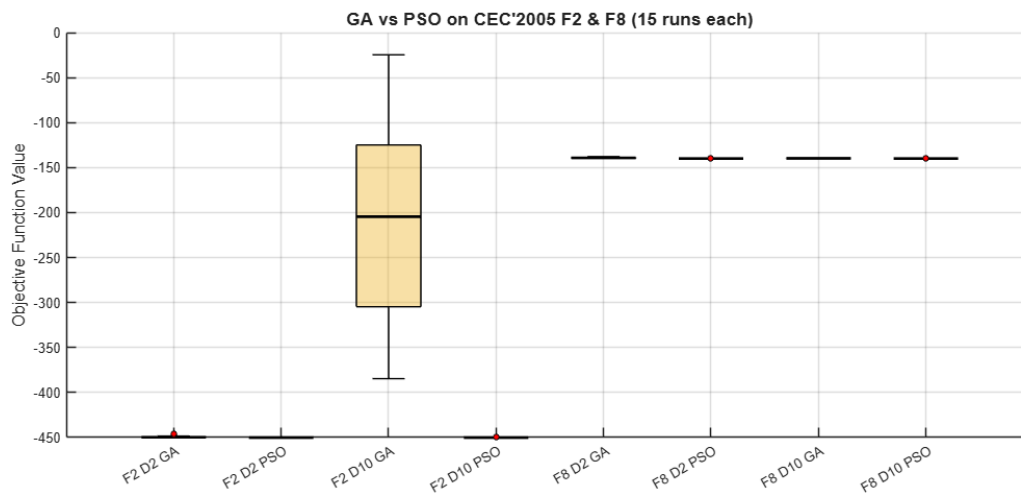
```

```

=====
=====
=====
key = sprintf('F%d_D%d', fid, D);
if isfield(results, key)
    r = results.(key);
    data_matrix = [data_matrix, r.ga_vals, r.pso_vals]; %#ok
    label_list{end+1} = sprintf('F%d D%d GA', fid, D); %#ok
    label_list{end+1} = sprintf('F%d D%d PSO', fid, D); %#ok
end
end
end

draw_boxplot(data_matrix, label_list);
ylabel('Objective Function Value','FontSize',12);
title('GA vs PSO on CEC'2005 F2 & F8 (15 runs
each)','FontSize',13,'FontWeight','bold');
grid on;

```



---- **Figure 2: Convergence Curves (mean $\pm$ std)**

This figure directly addresses the assignment requirement for "convergence graphs".
Each line = mean best-so-far across 15 runs.
Shaded band = ± 1 standard deviation.

```

figure('Name','Convergence Curves','NumberTitle','off','Position',[50 580
1200 500]);

```

```

plot_idx = 0;
for fi = 1:2
    fid = func_ids(fi);
    for di = 1:length(D_VALUES)
        D = D_VALUES(di);
        key = sprintf('F%d_D%d', fid, D);
        if ~isfield(results, key), continue; end
        r = results.(key);

```

```

=====
=====
=====
plot_idx = plot_idx + 1;
subplot(2, 2, plot_idx);
hold on;

% GA convergence: mean  $\pm$  std over 15 runs
n_iter = size(r.ga_conv, 2);
ga_m    = mean(r.ga_conv, 1);
ga_s    = std(r.ga_conv, 0, 1);
x_ga    = 1:n_iter;
% Shaded band
fill([x_ga, fliplr(x_ga)], ...
     [ga_m+ga_s, fliplr(ga_m-ga_s)], ...
     'b','FaceAlpha',0.15,'EdgeColor','none');
% Mean line
plot(x_ga, ga_m, 'b-', 'LineWidth', 2);

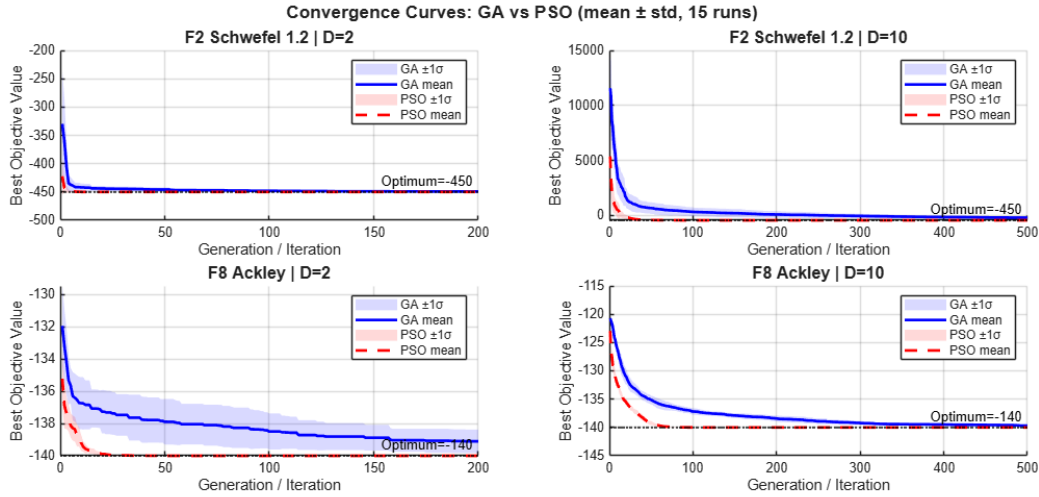
% PSO convergence
n_iter_p = size(r.pso_conv, 2);
pso_m    = mean(r.pso_conv, 1);
pso_s    = std(r.pso_conv, 0, 1);
x_pso    = 1:n_iter_p;
fill([x_pso, fliplr(x_pso)], ...
     [pso_m+pso_s, fliplr(pso_m-pso_s)], ...
     'r','FaceAlpha',0.15,'EdgeColor','none');
plot(x_pso, pso_m, 'r--', 'LineWidth', 2);

% Global optimum reference line
yline(r.optimum, 'k:', 'LineWidth', 1.5, ...
      'Label', sprintf('Optimum=%.0f',r.optimum));

xlabel('Generation / Iteration','FontSize',11);
ylabel('Best Objective Value','FontSize',11);
title(sprintf('%s | D=%d', r.func_name, r.D), ...
      'FontSize',12,'FontWeight','bold');
legend({'GA  $\pm 1\sigma$ ', 'GA mean', 'PSO  $\pm 1\sigma$ ', 'PSO mean'}, ...
      'Location','northeast','FontSize',9);
grid on;
end
end

sgtitle('Convergence Curves: GA vs PSO (mean  $\pm$  std, 15 runs)', ...
      'FontSize',13,'FontWeight','bold');

```



---- Figure 3: Per-Run Value Scatter

```

=====

figure('Name','Per-Run Values','NumberTitle','off','Position',[50 100 1000
420]);
colors = {'b','r'};

subplot(1,2,1); hold on; leg2 = {};
for di = 1:length(D_VALUES)
    D = D_VALUES(di);
    key = sprintf('F2_D%d', D);
    if isfield(results, key)
        r = results.(key);
        plot(1:N_RUNS, r.ga_vals, [colors{di} '-o'], 'MarkerSize',4);
        plot(1:N_RUNS, r.pso_vals, [colors{di} '--s'], 'MarkerSize',4);
        leg2{end+1} = sprintf('GA D=%d', D); %#ok
        leg2{end+1} = sprintf('PSO D=%d', D); %#ok
    end
end
legend(leg2,'Location','best'); grid on;
xlabel('Run'); ylabel('Best Value Found');
title('F2 Schwefel 1.2 - All 15 Runs','FontWeight','bold');

subplot(1,2,2); hold on; leg8 = {};
for di = 1:length(D_VALUES)
    D = D_VALUES(di);
    key = sprintf('F8_D%d', D);
    if isfield(results, key)
        r = results.(key);
        plot(1:N_RUNS, r.ga_vals, [colors{di} '-o'], 'MarkerSize',4);
        plot(1:N_RUNS, r.pso_vals, [colors{di} '--s'], 'MarkerSize',4);
        leg8{end+1} = sprintf('GA D=%d', D); %#ok
        leg8{end+1} = sprintf('PSO D=%d', D); %#ok
    end
end

```



```

end
legend(leg8,'Location','best'); grid on;
xlabel('Run'); ylabel('Best Value Found');
title('F8 Ackley - All 15 Runs','FontWeight','bold');
sgtitle('CEC2005: GA vs PSO Per-Run Final Values','FontSize',13);

disp(' ');
disp('=== Part 3 Complete ===');
disp('Three figures generated:');
disp(' Fig 1 - Box plots (distribution of 15 runs)');
disp(' Fig 2 - Convergence curves (mean ± std) ← new, required by brief');
disp(' Fig 3 - Per-run scatter');

```

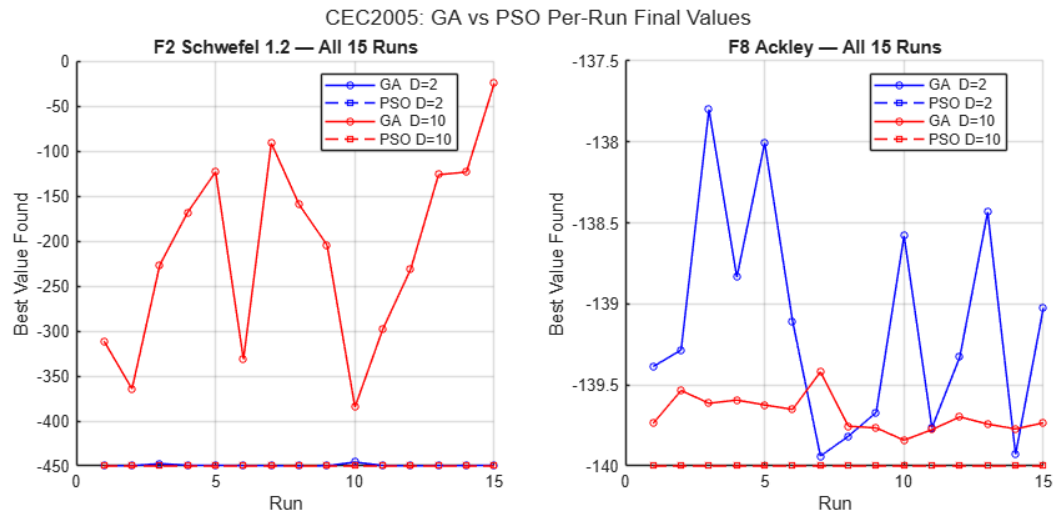
=== Part 3 Complete ===

Three figures generated:

Fig 1 - Box plots (distribution of 15 runs)

Fig 2 - Convergence curves (mean ± std) ← new, required by brief

Fig 3 - Per-run scatter



GA: returns best_val AND full convergence history

```

function [best_val, conv_hist] = run_custom_ga( ...
    fobj, D, lb, ub, popSize, nGens, pcross, pmut, eliteN)

% Initialise population
pop = lb + (ub-lb) .* rand(popSize, D);

```

```

=====
=====
=====
fitness = zeros(popSize, 1);
for i = 1:popSize
    fitness(i) = fobj(pop(i,:));
end

[best_val, ~] = min(fitness);
conv_hist      = zeros(1, nGens);

for gen = 1:nGens
    % Sort by fitness
    [fitness, idx] = sort(fitness, 'ascend');
    pop = pop(idx,:);

    if fitness(1) < best_val
        best_val = fitness(1);
    end
    conv_hist(gen) = best_val;    % <-- record best-so-far

    % Build next generation
    newPop = zeros(popSize, D);
    newPop(1:eliteN,:) = pop(1:eliteN,:);    % elitism

    for i = eliteN+1 : popSize
        p1 = tourn(fitness, popSize);
        p2 = tourn(fitness, popSize);

        % Uniform crossover (works correctly for D=2 unlike single-point)
        if rand < pcross
            mask = rand(1,D) > 0.5;
            child = pop(p1,:);
            child(mask) = pop(p2,mask);
        else
            child = pop(p1,:);
        end

        % Mutation (uniform reset within bounds)
        mut_mask = rand(1,D) < pmut;
        child(mut_mask) = lb(mut_mask) + ...
            (ub(mut_mask) -
lb(mut_mask)).*rand(1,sum(mut_mask));

        newPop(i,:) = max(lb, min(ub, child));
    end

    pop = newPop;
    for i = 1:popSize
        fitness(i) = fobj(pop(i,:));
    end
end

% Final best
best_val = min([best_val; fitness]);
end

```

=====

=====

===

GA run 1: -311.2734
GA run 2: -364.3602
GA run 3: -226.4484
GA run 4: -168.1035
GA run 5: -123.0870
GA run 6: -331.4198
GA run 7: -91.2704
GA run 8: -159.6390
GA run 9: -204.5368
GA run 10: -384.5450
GA run 11: -297.9480
GA run 12: -231.2805
GA run 13: -126.3203
GA run 14: -123.3894
GA run 15: -24.6896

GA run 1: -139.3862
GA run 2: -139.2842
GA run 3: -137.7995
GA run 4: -138.8340
GA run 5: -138.0055
GA run 6: -139.1060
GA run 7: -139.9389
GA run 8: -139.8162
GA run 9: -139.6692
GA run 10: -138.5790
GA run 11: -139.7641
GA run 12: -139.3270
GA run 13: -138.4321
GA run 14: -139.9279
GA run 15: -139.0238

GA run 1: -139.7368
GA run 2: -139.5339
GA run 3: -139.6129
GA run 4: -139.5930
GA run 5: -139.6234
GA run 6: -139.6508
GA run 7: -139.4189
GA run 8: -139.7543
GA run 9: -139.7650
GA run 10: -139.8407
GA run 11: -139.7757
GA run 12: -139.6966
GA run 13: -139.7415
GA run 14: -139.7712
GA run 15: -139.7335


```

=====
=====
=====
pbest(improved,:) = pos(improved,:);
pbest_f(improved) = fitness(improved);

% Update global best
[iter_best, idx] = min(pbest_f);
if iter_best < gbest_f
    gbest_f = iter_best;
    gbest    = pbest(idx,:);
end

conv_hist(iter) = gbest_f; % <-- record best-so-far
end

best_val = gbest_f;
end

PSO run  1: -450.0000
PSO run  2: -450.0000
PSO run  3: -450.0000
PSO run  4: -450.0000
PSO run  5: -450.0000
PSO run  6: -450.0000
PSO run  7: -450.0000
PSO run  8: -450.0000
PSO run  9: -450.0000
PSO run 10: -450.0000
PSO run 11: -450.0000
PSO run 12: -450.0000
PSO run 13: -450.0000
PSO run 14: -450.0000
PSO run 15: -450.0000

PSO run  1: -450.0000
PSO run  2: -450.0000
PSO run  3: -450.0000
PSO run  4: -450.0000
PSO run  5: -450.0000
PSO run  6: -450.0000
PSO run  7: -450.0000
PSO run  8: -450.0000
PSO run  9: -450.0000
PSO run 10: -450.0000
PSO run 11: -450.0000
PSO run 12: -450.0000
PSO run 13: -450.0000
PSO run 14: -450.0000
PSO run 15: -450.0000

PSO run  1: -140.0000
PSO run  2: -140.0000
PSO run  3: -140.0000
PSO run  4: -140.0000
PSO run  5: -140.0000
PSO run  6: -140.0000
PSO run  7: -140.0000

```

```
=====
=====
=====
PSO run 8: -140.0000
PSO run 9: -140.0000
PSO run 10: -140.0000
PSO run 11: -140.0000
PSO run 12: -140.0000
PSO run 13: -140.0000
PSO run 14: -140.0000
PSO run 15: -140.0000
```

```
PSO run 1: -140.0000
PSO run 2: -140.0000
PSO run 3: -140.0000
PSO run 4: -140.0000
PSO run 5: -140.0000
PSO run 6: -140.0000
PSO run 7: -140.0000
PSO run 8: -140.0000
PSO run 9: -140.0000
PSO run 10: -140.0000
PSO run 11: -140.0000
PSO run 12: -140.0000
PSO run 13: -140.0000
PSO run 14: -140.0000
PSO run 15: -140.0000
```

```
=====
```

```
=====
```

HELPERS

```
=====
```

```
=====
```

```
function idx = tourn(fitness, popSize)
%TOURN Binary tournament selection.
    a = randi(popSize);
    b = randi(popSize);
    idx = a;
    if fitness(b) < fitness(a), idx = b; end
end

function draw_boxplot(data_matrix, labels)
%DRAW_BOXPLOT Manual box-and-whisker plot (no Stats Toolbox).
    if size(data_matrix,1) < size(data_matrix,2)
        data_matrix = data_matrix';
    end
    nGroups = size(data_matrix, 2);
    colors_box = lines(nGroups);
    hold on;
    for g = 1:nGroups
        col = sort(data_matrix(~isnan(data_matrix(:,g)), g));
```

```

=====
=====
=====

q1 = prctile_manual(col, 25);
q2 = prctile_manual(col, 50);
q3 = prctile_manual(col, 75);
iqr_ = q3 - q1;
wlo = max(min(col), q1 - 1.5*iqr_);
whi = min(max(col), q3 + 1.5*iqr_);
out = col(col < wlo | col > whi);
xc = g; bw = 0.3;

patch([xc-bw, xc+bw, xc+bw, xc-bw], [q1,q1,q3,q3], ...
      colors_box(g,:), 'FaceAlpha',0.4,'EdgeColor','k');
line([xc-bw,xc+bw], [q2,q2], 'Color','k','LineWidth',2);
line([xc,xc], [q3,whi], 'Color','k');
line([xc,xc], [q1,wlo], 'Color','k');
line([xc-bw/2,xc+bw/2], [whi,whi], 'Color','k');
line([xc-bw/2,xc+bw/2], [wlo,wlo], 'Color','k');
if ~isempty(out)
    plot(xc*ones(size(out)), out,
'ko','MarkerSize',4,'MarkerFaceColor','r');
end
end
set(gca,'XTick',1:nGroups,'XTickLabel',labels);
xtickangle(30); xlim([0, nGroups+1]);
end

function p = prctile_manual(x, pct)
%PRCTILE_MANUAL Linear-interpolation percentile (no Stats Toolbox).
x = sort(x(:));
n = numel(x);
idx = (pct/100)*(n-1) + 1;
lo = floor(idx); hi = ceil(idx);
if lo == hi
    p = x(lo);
else
    p = x(lo) + (idx-lo)*(x(hi)-x(lo));
end
end

GA run 1: -449.8992
GA run 2: -449.6236
GA run 3: -447.4060
GA run 4: -449.1462
GA run 5: -449.1880
GA run 6: -449.6172
GA run 7: -449.6302
GA run 8: -449.9693
GA run 9: -449.7579
GA run 10: -445.7167
GA run 11: -449.4408
GA run 12: -449.8856
GA run 13: -449.5205
GA run 14: -449.6572
GA run 15: -449.0696

```

=====

=====

=====

Published with MATLAB® R2026a